Predicting Stack Overflow Tags from Question Titles

Metis Data Science Bootcamp, Project 3, February 2021

Garreth Cline

Slides remade in October 2026 with AI help. Every result comes from the original 2021 notebook.

Code and notebook: github.com/garrethcline/Predicting-Tags-from-StackOverflow-Using-Title-Data-

The goal

Recommend tags for a Stack Overflow question from its title alone.

- Suggest tags to someone while they write a question.
- Add tags to old posts or other text that has none.
- A question can have several tags, so for each title the model makes 100 yes or no calls, one for each of the 100 most common tags.

Title	Tags kept
Convert Decimal to Double?	c#
How do I calculate someone's age in C#?	c#, .net, datetime
Calculate relative time in C#	c#, datetime
Determine a User's Timezone	javascript, html

Examples from the data. Tags outside the 100 most common were dropped.

The data

Stack Overflow's public data dump (2010 version), loaded as a SQL Server database from brentozar.com

1,000,000 posts exported → **28,183** score above 50 → **27,666** over 10,000 views → **25,352** with a top-100 tag

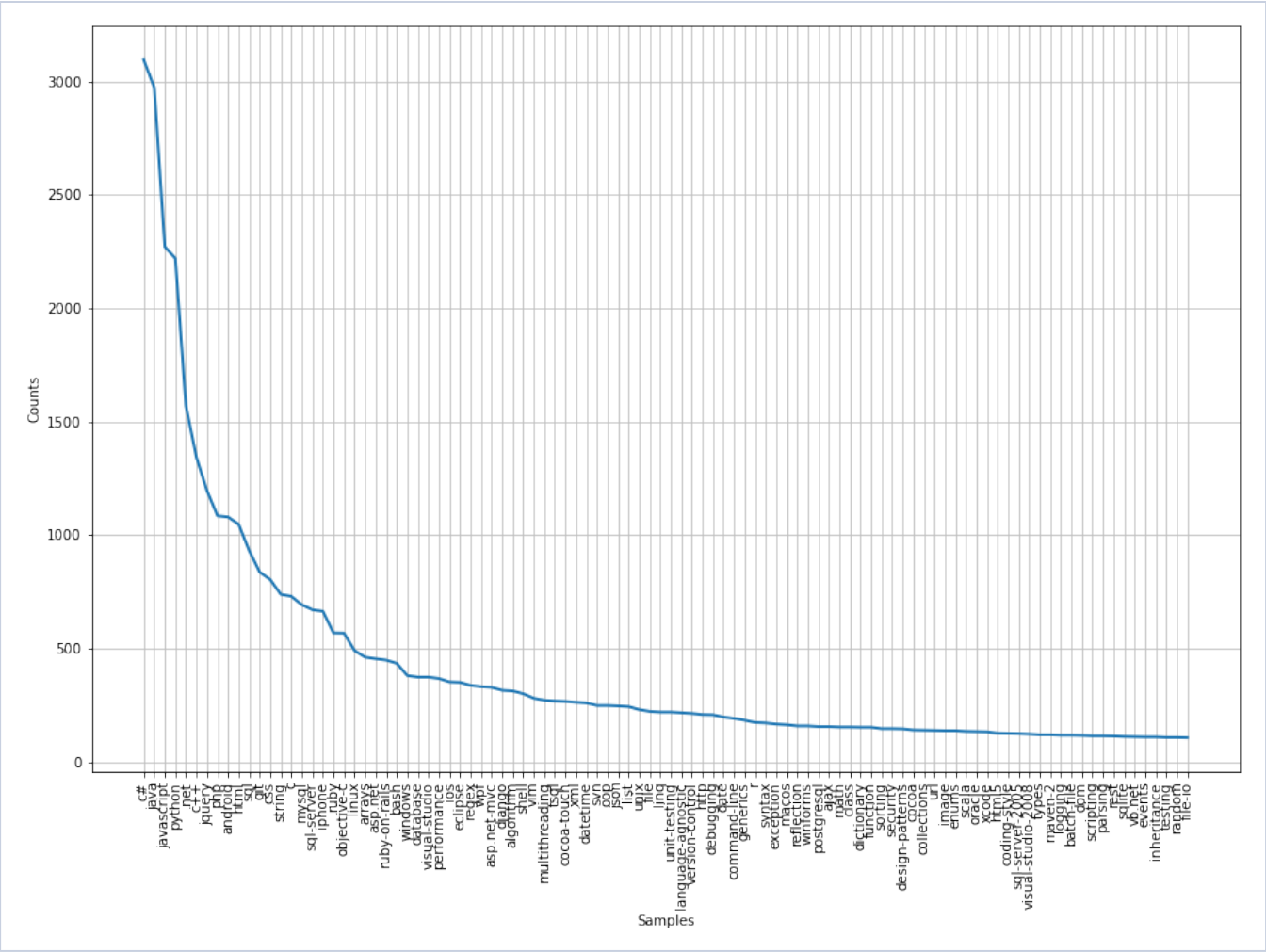
In SQL Server

- Removed commas from titles and tags, and the angle brackets around tags, so the CSV would load cleanly
- Deleted posts with no title
- Exported the posts to CSV with SQL Server Management Studio

In Python

- Counted 82,516 tag uses across 6,884 different tags and kept the 100 most common
- Cleaned the titles: removed punctuation, reduced verbs to their base form, removed common words
- Scored each title with TF-IDF over the 1,000 most frequent words
- Held out 20% of the questions (5,071) as a test set

The 100 most common tags



How often each tag appears in the 27,666 questions left after the score and view filters.

c# is the most common, on about 3,100 questions, followed by java, javascript and python. By the 100th tag (file-io) the count is close to 100.

So c# has about 30 times as many examples to learn from as file-io.

Chart from the 2021 notebook.

Twelve models on the same test set

Each model ran as 100 yes or no classifiers, one per tag (one-vs-rest), with default settings.

Model	Jaccard score	Hamming loss
PassiveAggressive	48.5	1.19%
LinearSVC (chosen)	**47.8**	**1.02%**
Random forest, 3 runs	45.8 to 45.9	1.07 to 1.08%
SGD	45.7	1.02%
Decision tree	43.1	1.41%
Perceptron	42.7	1.70%
Logistic regression	41.1	1.09%
k-nearest neighbors	24.2	1.42%
Multinomial naive Bayes	23.2	1.35%
Dummy (random guesses)	2.6	3.14%

Reading the scores

Jaccard score

For each test question: the tags both predicted and correct, divided by all tags that were predicted or correct. Averaged and shown out of 100. Higher is better.

Hamming loss

The share of all yes or no calls that were wrong. Lower is better.

Even the dummy model gets about 97% of calls right, since most tags don't apply to a given question. The Jaccard score shows the differences better.

The 2021 code labeled the three random forests 10, 100 and 1,000 trees, but all three were set to 1,000, so they share a row. The dummy model guessed each tag at random, at that tag's rate in the training data.

Final model: LinearSVC

I tuned LinearSVC, which had the lowest Hamming loss and the second-highest Jaccard score.

A grid search tried C = 1, 10, 100 and 1,000 with 100-fold cross-validation: 400 fits, about four minutes on a 32-thread computer.

The best value was C = 1, the default, so the tuned model scored the same as the untuned one.

100 folds was more than needed. The notebook notes that 15 folds gave similar results with much less load.

47.8

Jaccard score

Dummy model: 2.6

1.02%

Hamming loss

Dummy model: 3.14%

85.5%

Precision

Share of predicted tags that were correct

46.0%

Recall

Share of actual tags the model found

Precision and recall were added in 2026, from the notebook's 100 per-tag confusion matrices: 3,842 tags found, 652 predicted wrongly, 4,508 missed.

Where it worked and where it didn’t

Highest recall (tags with 25 or more test questions)

Tag	Found	Recall
json	39 of 44	89%
git	148 of 170	87%
eclipse	54 of 65	83%
django	55 of 69	80%
svn	39 of 49	80%
sqlite	23 of 29	79%
ruby-on-rails	77 of 99	78%
vim	45 of 59	76%

The eight most common tags

Tag	Found	Recall
c#	183 of 597	31%
java	342 of 595	57%
javascript	258 of 466	55%
python	309 of 450	69%
.net	77 of 302	25%
c++	66 of 282	23%
php	148 of 236	63%
android	176 of 234	75%

The best tags are names of tools that tend to appear in titles. Broad topic tags did worst: 11 tags were never predicted correctly, including oop, class, file, math, coding-style and language-agnostic.

What the model learned

The eight title words with the most weight for each tag in the final model, strongest first

Tag	Words
git	git, github, commit, branch, repo, repository, remote, master
android	android, activity, webview, edittext, textview, intent, listview, emulator
python	python, matplotlib, numpy, dict, django, module, none, modules
java	java, jpa, spring, junit, jvm, hibernate, servlet, arraylist
css	css, border, divs, div, font, span, align, overflow
sql	sql, procedure, oracle, inner, query, t-sql, on, by
iphone	iphone, uiscrollview, uitableview, uiview, uibutton, uiimage, uilabel, ios
c++	std, cc, boost, c++, stl, virtual, qt, template

For most tags the strongest word is the tag's own name. The exception here is c++, where it is "std"; the next slide explains why.

Why c#, c++ and c scored low

The punctuation cleanup kept a tag name only when it matched exactly, in lower case. When a title said “C#” or “C++”, the symbols were stripped and only “C” was left.

The TF-IDF step ignored one-letter words, so the “C” was dropped too. For those titles the model never saw the language name.

The same cleanup turned “C/C++” into “cc”, which became the strongest word for the c tag.

The fix is to match tag names in any case before removing punctuation.

Title in the data

How do I calculate someone’s age in C#?

After cleanup (notebook output)

How do I calculate someone s age in C

Tag	Test questions	Recall
c#	597	31%
c++	282	23%
c	151	10%

Found in 2026 while reviewing the 2021 code.

Limitations and what I would change

- Add the question body. I left it out in 2021 because it took much more memory and I wanted more rows, and titles often don’t name the tool.
- Fix the cleanup so C# and C++ survive in any case, and make the common-word filter ignore case (capitalized words like “How” got through).
- Tune a separate cutoff for each tag, trading some precision (85.5%) for more recall (46.0%).
- The data ends in 2010 and covers only popular questions (score above 50, over 10,000 views), so results may differ on typical questions.
- Only the 100 most common tags were modeled; the other 6,784 were left out.